

```

1: function CS_TSSOS_FIRST(pop, x, d, keyword arguments)
2:   ▷ obtain more efficient way to represent monomials
3:   n, supp, coe ← polys_info (pop, x)
4:
5:   ▷ Merge monomials with same support in objective
6:   supp[1], coe[1] ← resort (supp[1], coe[1])
7:
8:   if clique is not specified then
9:     cliques ← clique_decomp (n, m, numeq, dc, supp)
10:
11:   ▷ Group Constraints according to variables and cliques
12:   I, J, ncc ← assign_constraint (m, numeq, supp, cliques, cql)
13:
14:   ▷ Determine max degree of monomials in subproblem for each clique
15:   if d is “min” then
16:     rlorder ← max degree in I, J for each clique
17:   else
18:     rlorder ← d for all cliques
19:
20:   if custom basis not specified then
21:     ▷ If Correlative Sparse, need to loop over all cliques
22:     basis ← get_sbasis (cliques, rlorder)
23:     ▷ Why is this sparse?
24:     hbasis ← get_sbasis ()
25:   if Term Sparsity is required then
26:     ksupp ← supp + square of all terms in supp
27:     ▷ Apply Term Sparsity by chordal extension, We can improve this
28:     blocks, eblocks, cl, blocksize ← get_blocks (I, J, supp, cliques, cql, ...)
29:     opt, ksupp, moment, GramMat ... ← solvesdp (n,m,...)
30:     ▷ Construct the data structure for higher order improvement
31:     data ← mcpop_data (n, nb, m, numeq, ...)
32:     ▷ Why is this step necessary?
33:     if solution is true then
34:       sol, gap, data.flag ← approx_sol (n,m,...)
35:     return opt, sol, data

```

```

1: function CS_NCTSSOS_FIRST(pop, x, d, keyword arguments)
2:   ▷ obtain more efficient way to represent monomials
3:   n, supp, coe ← polys_info (pop, x)
4:
5:   ▷ Merge terms with canonically same support in objective
6:   if obj is “trace” then
7:     obj_supp, obj_coe ← cyclic_canon (obj_supp, obj_coe)
8:   else if obj is “eigen” then
9:     obj_supp, obj_coe ← sym_canon (obj_supp, obj_coe)
10:
11:   ▷ Decompose into cliques, we can come in
12:   if Require Correlative Sparsity then
13:     cliques ← clique_decomp (n, m, numeq, dc, supp)
14:

```

```

15:   ▷ Group Constraints according to variables and cliques
16:   J , ncc ← assign_constraint (m, supp, cliques, cql)
17:
18:   ▷ Get basis for objective and constraints
19:   for do
20:     clique in cliques
21:     objective basis ← get_ncbasis (cliquesize, d, clique)
22:     constraint basis ← get_ncbasis (cliquesize, d, clique)
23:     if variables contains commutative ones then
24:       basis ← simplify (objective basis, constraint basis)
25:   ▷ Reduce the support by considering commutative portion of variables
26:   tsupp ← reduce!. (tsupp, obj=obj, par)
27:   ▷ Apply Term Sparsity by chordal extension, We can improve this
28:   blocks, eblocks, cl, blocksize ← get_blocks (I, J , supp, cliques, cql, ...)
29:   opt, ksupp, moment, GramMat ... ← solvesdp (n,m,...)
30:   ▷ Construct the data structure for higher order improvement
31:   data ← mcpop_data (n, nb, m, numeq, ...)
32:   ▷ Why is this step necessary?
33:   if solution is true then
34:     sol, gap, data.flag ← approx_sol (n,m,...)
35:   return opt, sol, data

```